Marco de explicabilidad basado en SHAP y LIME para mejorar la interpretabilidad de las alertas de detección de intrusiones en SOC que monitorean redes IoT a nivel internacional, 2026

Jorge Luis Berrocal Barrientos

Facultad de Ingeniería Eléctrica y Electrónica, Universidad Nacional de Ingeniería, Lima, Perú

Resumen

Los Centros de Operaciones de Seguridad (SOC) reciben más alertas de las que pueden revisar, y los detectores basados en aprendizaje automático las emiten sin explicar qué características del tráfico las motivaron. Este estudio propone y evalúa un marco de explicabilidad que integra TreeSHAP y LIME a un detector Random Forest que clasifica ventanas de tráfico de malware IoT por familia (Mirai, DarkNexus y Gafgyt), con datos del dataset internacional CIC-YNU-IoTMal 2026. Con un diseño cuasi-experimental de medidas repetidas, cada una de 390 alertas se explicó bajo tres condiciones: la importancia global del modelo (línea base), TreeSHAP y LIME. La interpretabilidad se midió por la fidelidad (caída de probabilidad e inversión de la decisión al neutralizar las cinco características principales), la estabilidad (índice de Jaccard entre ejecuciones y ante perturbaciones) y la parsimonia (concentración de la atribución), y las hipótesis se contrastaron con pruebas de Friedman, Q de Cochran y Wilcoxon con corrección de Holm. El detector alcanzó una exactitud de 0,886 y un F1 macro de 0,553, limitado por las familias minoritarias. TreeSHAP produjo las explicaciones más concisas (mediana de NC80 = 7 frente a 9 de la línea base; $r = -0{,}70$) y LIME las menos concisas (NC80 = 12). Ningún método superó a la línea base en fidelidad (TID@k de 41,0 %, 41,8 % y 41,3 %; $Q = 3{,}5$, $p = 0{,}174$), y ninguno alcanzó la estabilidad de 0,80 (RP de 0,67 en TreeSHAP; EE de 0,50 y RP de 0,43 en LIME). Se acepta parcialmente H.E.3 (solo TreeSHAP) y no se aceptan H.E.1 ni H.E.2. Los resultados aportan evidencia cuantitativa y reproducible para elegir el método de explicación de alertas en un SOC.

Palabras clave: inteligencia artificial explicable, SHAP, LIME, detección de intrusiones, malware IoT, centro de operaciones de seguridad.

Ideas destacadas

- Se integra TreeSHAP y LIME a un detector Random Forest de alertas de malware IoT y se evalúan contra una línea base global.
- La calidad de cada explicación se mide por su fidelidad, estabilidad y parsimonia con pruebas pareadas no paramétricas.
- TreeSHAP es determinista y más conciso que la línea base; LIME es inestable y disperso, y ninguno supera a la línea base en fidelidad.

1. Introducción

Los Centros de Operaciones de Seguridad (SOC) dependen de sistemas de detección de intrusiones que emiten alertas a un ritmo mayor del que sus analistas pueden atender. En una encuesta a 2 000 profesionales de diez países, los equipos SOC declararon recibir en promedio 4 484 alertas diarias y dejar sin atender el 67 % de ellas (Vectra AI, 2023); otro estudio con 1 000 miembros de SOC encontró que solo llegan a revisar el 49 % de las alertas de una jornada (IBM Security, 2023). La validación manual de alertas, muchas de ellas falsos

positivos, conduce al agotamiento y a la desensibilización de los analistas, un efecto que se agrava cuando las alertas no explican su origen (Alahmadi et al., 2022).

La presión es mayor en las redes del Internet de las cosas (IoT), formadas por dispositivos heterogéneos, con recursos limitados y a menudo expuestos con credenciales por defecto. La botnet Mirai llegó a infectar cerca de 600 000 dispositivos y demostró la capacidad de estos equipos para lanzar ataques de denegación de servicio a gran escala (Antonakakis et al., 2017), mientras que honeypots como IoTPOT siguen capturando muestras de malware IoT procedentes de todo el mundo (Pa et al., 2016). Los detectores basados en aprendizaje automático alcanzan una alta exactitud frente a estas amenazas, pero los modelos de ensamble y las redes profundas que emplean son opacos para el usuario (Barredo Arrieta et al., 2020).

El problema que aborda este estudio es la baja interpretabilidad de las alertas, entendida como el grado en que un observador puede comprender la causa de una decisión (Miller, 2019). Una alerta que solo informa una etiqueta o una probabilidad obliga al analista a reconstruir la evidencia por su cuenta. Los métodos de explicación locales post hoc, como SHAP (Lundberg & Lee, 2017) y LIME (Ribeiro et al., 2016), asocian a cada predicción las características que más contribuyeron a ella; sin embargo, la calidad de sus explicaciones varía según la métrica, el modelo y los datos, y ningún método resulta superior en todos los criterios (Arreche et al., 2024; Hermosilla et al., 2025; Munilla & Khammas, 2026).

La mayoría de estas comparaciones usa conjuntos de datos de laboratorio de una sola institución, no contrasta las explicaciones locales con la información global de la que el analista ya dispone y rara vez aplica pruebas estadísticas sobre las mismas alertas. En la búsqueda realizada no se encontró ningún estudio que compare SHAP y LIME con métricas cuantitativas de fidelidad, estabilidad y parsimonia sobre CIC-YNU-IoTMal 2026, un dataset internacional de tráfico de malware IoT capturado en cuatro arquitecturas (Dadkhah et al., 2026). El aporte de este trabajo es triple: (a) un marco que integra TreeSHAP y LIME a un detector Random Forest de alertas por familia de malware; (b) un protocolo de evaluación con línea base global, tres dimensiones de calidad y contrastes pareados; y (c) un experimento reproducible con código y datos públicos.

El objetivo general es desarrollar un marco de explicabilidad basado en SHAP y LIME que mejore la interpretabilidad de las alertas de detección de intrusiones en SOC que monitorean redes IoT a nivel internacional. Los objetivos específicos son analizar en qué medida el marco mejora la fidelidad de las explicaciones (O.E.1), evaluar en qué medida asegura su estabilidad (O.E.2) y determinar en qué medida mejora su parsimonia (O.E.3). Se busca que cada alerta llegue con una justificación verificable que reduzca el esfuerzo de triaje.

2. Revisión de la literatura

Los trabajos que explican detectores de intrusiones IoT basados en árboles muestran la utilidad de SHAP, aunque sin medir la calidad de sus explicaciones. Le et al. (2022) entrenaron árboles de decisión y Random Forest sobre tres conjuntos IoT y los explicaron con TreeSHAP mediante mapas de calor globales y gráficos de decisión locales, sin métricas cuantitativas de calidad. Alani y Miri (2022) emplearon SHAP con TreeExplainer para justificar un conjunto universal de seis características seleccionadas con Random Forest, validado también sobre IoT-23, pero su evaluación se centró en la exactitud del detector y no en la de las explicaciones.

Otro grupo de estudios compara SHAP y LIME con métricas explícitas. Arreche et al. (2024) propusieron E-XAI, un marco que evalúa ambos métodos sobre siete modelos y tres conjuntos de datos de intrusión con seis métricas: exactitud descriptiva, dispersión,

estabilidad, eficiencia, robustez y completitud. Hermosilla et al. (2025) compararon ambos explicadores sobre XGBoost y TabNet en UNSW-NB15 con métricas de fidelidad y estabilidad basadas en el índice de Jaccard, y señalaron el alto costo computacional de KernelSHAP por instancia. Munilla y Khammas (2026) evaluaron KernelSHAP y LIME en detectores IoT bajo ataques adversariales DeepFool: SHAP obtuvo mayor completitud y alineación de las características, mientras que LIME mostró mayor estabilidad en el ranking de las principales características, aunque con más atribuciones espurias.

La Tabla 1 resume estos antecedentes. En conjunto, revelan tres vacíos: no hay un método ganador en todas las dimensiones; las métricas no están estandarizadas, lo que exige definirlas operacionalmente; y ningún estudio contrasta las explicaciones locales con una línea base global sobre datos recientes de malware IoT de origen internacional. Este trabajo aborda los tres vacíos.

Tabla 1

Comparación con metodologías de estudios similares

Estudio	Modelo	Explicadores	Datos	Métricas de explicación
Le et al. (2022)	DT, RF	TreeSHAP	IoTID20, NF-BoT-IoT-v2, NF-ToN-IoT-v2	Cualitativas (sin métricas de calidad)
Alani y Miri (2022)	DT, RF	TreeSHAP	TON_IoT, IoT-23	No evalúa su calidad
Arreche et al. (2024)	Siete modelos	SHAP, LIME	Tres datasets de IDS	Exactitud descriptiva, dispersión, estabilidad, eficiencia, robustez, completitud
Hermosilla et al. (2025)	XGBoost, TabNet	SHAP, LIME	UNSW-NB15	Fidelidad, estabilidad (Jaccard), tiempo
Munilla y Khammas (2026)	RF, DNN, CNN, LSTM	KernelSHAP, LIME	BoT-IoT, Edge-IIoT, N-BaIoT	Dispersión, completitud, robustez
Este estudio	RF	MDI (base), TreeSHAP, LIME	CIC-YNU-IoTMal 2026	Fidelidad, estabilidad, parsimonia, costo

3. Metodología

3.1 Diseño de la investigación

El estudio tiene enfoque cuantitativo, es de tipo aplicado y de nivel explicativo, con un diseño cuasi-experimental, transversal y de medidas repetidas. La unidad de análisis es la *alerta*, definida como una ventana de tráfico del conjunto de prueba a la que el detector asigna una familia de malware. Cada alerta se observa bajo tres condiciones de explicación: C0, la importancia global del Random Forest por disminución media de la impureza (MDI), que representa la información de la que dispone el analista sin explicaciones locales (pre-test); C1, la explicación local de TreeSHAP; y C2, la explicación local de LIME (post-test). Como las tres condiciones se aplican a las mismas alertas y al mismo modelo, las diferencias se atribuyen al método de explicación. El diseño retoma la lógica comparativa de los estudios de la Tabla 1, pero añade la línea base global y contrastes pareados. Se contrastan tres hipótesis específicas: la fidelidad de C1 y C2 supera a la de C0 (H.E.1); la estabilidad de C1 y C2 es alta, con un índice de Jaccard de al menos 0,80 (H.E.2); y la parsimonia de C1 y C2 supera a la de C0 (H.E.3).

3.2 Descripción del dataset

Se utilizó CIC-YNU-IoTMal 2026 (Dadkhah et al., 2026), elaborado por el Canadian Institute for Cybersecurity de la University of New Brunswick (Canadá) y la Yokohama National University (Japón), con apoyo del NICT y de la Universidad de Kobe. Las muestras de malware fueron capturadas a nivel mundial por los honeypots IoTPOT (Pa et al., 2016) y ejecutadas en un sandbox con máquinas virtuales QEMU y OpenWrt en cuatro arquitecturas de procesador. Para cada ejecución, el dataset registra tres modalidades sincronizadas: tráfico de red (PCAP procesado en ventanas con 39 características del esquema de CICIoT2023; Neto et al., 2023), actividad del sistema (SAR) y llamadas al sistema (STRACE). Este estudio usa la modalidad de red de la arquitectura ARM, que es la que redistribuye Kaggle (archivo `pcap.csv`, 737 651 ventanas y 5 599 binarios; Tabla 2), con etiquetas de familia por binario; el conjunto completo del CIC suma 3 167 325 ventanas en las cuatro arquitecturas. Se descartaron las etiquetas Unknown, Generic y Other. Se eligió este conjunto por su carácter internacional, su publicación en 2026, la diversidad de arquitecturas y su artículo de referencia revisado por pares.

Tabla 2

Ventanas de tráfico de red y binarios por familia en la partición ARM de CIC-YNU-IoTMal 2026

Familia	Ventanas	Binarios	Uso en el estudio
Mirai	507 772	2 795	Incluida
DarkNexus	44 156	91	Incluida
Gafgyt	10 450	5	Incluida
Unknown y Generic	122 670	724	Excluidas (sin familia)
Benign	51 791	1 980	Excluida (generada con LLM)
Tsunami y Agent	812	4	Excluidas (menos de 5 binarios)
Total	737 651	5 599	

Nota. Elaborado con src/01_explorar_dataset.py (results/exploracion/resumen_exploracion.md).

3.3 Preprocesamiento

Sobre los datos crudos se aplicaron cinco pasos: (a) verificación de integridad los archivos descargados; (b) normalización de las etiquetas de familia y selección de Mirai, DarkNexus y Gafgyt; (c) eliminación de identificadores (hash, rutas, marcas de tiempo, arquitectura), de columnas constantes o truncadas y de la variable IAT, por el riesgo de que actúe como un atajo ligado al momento de la captura y no al comportamiento del tráfico (Arp et al., 2022); (d) descarte de valores infinitos o nulos y de vectores duplicados; y (e) partición 70/30 agrupada por el hash del binario y estratificada por familia, de modo que ninguna muestra de malware aparezca en entrenamiento y prueba a la vez. Esta partición evita la fuga de información que produce una división aleatoria de ventanas, uno de los errores más frecuentes en la evaluación de detectores (Arp et al., 2022). De las 562 378 ventanas de las tres familias se eliminaron 281 con valores infinitos o nulos y 131 377 duplicadas; SMTP, SSH e IRC resultaron constantes, por lo que quedaron 35 características. Para ejecutar el experimento en una computadora portátil, Mirai se redujo de 405 573 a 149 989 ventanas mediante un submuestreo de binarios completos, y el desbalance restante se compensó con pesos de clase. El entrenamiento quedó con 130 954 ventanas (848 binarios) y la prueba con 44 182 (364 binarios); Gafgyt solo aporta cinco binarios (tres y dos), una limitación del dataset que se discute en la Sección 6. No se escalaron las variables, porque los árboles son invariantes ante transformaciones monótonas.

3.4 Procedimiento experimental

La Figura 1 resume el procedimiento. Las alertas del conjunto de prueba se muestrearon de forma estratificada por familia, con 130 por estrato ($n = 390$); este tamaño supera las 384 instancias necesarias para estimar proporciones con 95 % de confianza y 5 % de error, y da el mismo peso a las tres familias. Cada alerta se explicó con las tres condiciones usando $k = 5$ características principales, un compromiso entre la capacidad de lectura del analista y la cobertura de la decisión. LIME se ejecutó $R = 10$ veces con semillas distintas; TreeSHAP, que es determinista, una sola vez. Todo se ejecutó en CPU con Python 3 (scikit-learn, shap, lime y SciPy).

Figura 1

Procedimiento experimental del marco de explicabilidad



3.5 Métricas de evaluación

Sea $\hat{c}$ la familia que el detector predice para la alerta $\mathbf{x}$, $f(\mathbf{x}) = p(\hat{c} \mid \mathbf{x})$ su probabilidad, $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^d$ el vector de atribuciones de una condición y $T_k(\mathbf{x})$ el conjunto de sus k características con mayor $|\phi_j|$.

Fidelidad. Mide si las características señaladas sostienen realmente la decisión, siguiendo la lógica de perturbación por eliminación (Samek et al., 2017) adaptada a la detección de intrusiones (Arreche et al., 2024). Sea $\tilde{\mathbf{x}}^{(k)}$ la alerta en la que las características de $T_k(\mathbf{x})$ se reemplazan por su mediana en el conjunto de entrenamiento. Se definen la caída media de probabilidad y la tasa de inversión de la decisión:

$$\Delta p@k = \frac{1}{n}\sum_{i=1}^{n}\left[f(\mathbf{x}_i) - f\left(\tilde{\mathbf{x}}_i^{(k)}\right)\right] \tag{1}$$

$$TID@k = \frac{100}{n}\sum_{i=1}^{n}\mathbb{1}\left[\operatorname{argmax}_c\, p\left(c \mid \tilde{\mathbf{x}}_i^{(k)}\right) \neq \hat{c}_i\right] \tag{2}$$

Valores más altos indican mayor fidelidad: neutralizar lo que la explicación considera importante debilita o cambia la decisión.

Estabilidad. Se cuantifica con el índice de Jaccard entre conjuntos top-k, $J(A,B) = |A \cap B|/|A \cup B|$ (Hermosilla et al., 2025). La estabilidad entre ejecuciones promedia todos los pares de las R repeticiones:

$$EE = \frac{1}{n}\sum_{i=1}^{n}\frac{2}{R(R-1)}\sum_{r<s} J\left(T_k^{(r)}(\mathbf{x}_i),\, T_k^{(s)}(\mathbf{x}_i)\right) \tag{3}$$

La robustez compara la explicación de $\mathbf{x}$ con la de una versión perturbada $\mathbf{x}' = \mathbf{x} + \varepsilon\,\boldsymbol{\sigma} \odot \mathbf{u}$, con $\mathbf{u} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, $\boldsymbol{\sigma}$ la desviación estándar de cada característica en entrenamiento y $\varepsilon = 0{,}01$; solo se aceptan perturbaciones que no cambian la familia predicha (Munilla & Khammas, 2026):

$$RP = \frac{1}{n}\sum_{i=1}^{n} J\left(T_k(\mathbf{x}_i),\, T_k(\mathbf{x'}_i)\right) \tag{4}$$

Parsimonia. Mide cuán concentrada está la atribución (Arreche et al., 2024). Con las atribuciones ordenadas $|\phi_{(1)}| \geq \cdots \geq |\phi_{(d)}|$:

$$NC80(\mathbf{x}) = \min\left\{m: \frac{\sum_{j=1}^{m}|\phi_{(j)}|}{\sum_{j=1}^{d}|\phi_j|} \geq 0{,}80\right\} \tag{5}$$

$$CT3(\mathbf{x}) = 100 \cdot \frac{\sum_{j=1}^{3}|\phi_{(j)}|}{\sum_{j=1}^{d}|\phi_j|} \tag{6}$$

Un $NC80$ menor y un $CT3$ mayor indican explicaciones más concisas.

Desempeño y costo (variables de control). El detector se evalúa con el F1 macro y con la tasa de falsos positivos de cada familia frente al resto, $TFP = FP/(FP + VN)$. El costo de cada explicador se mide con el tiempo medio por alerta, $TME = \frac{1}{n}\sum_i t_i$, y con la memoria pico. La concordancia entre C1 y C2 se mide con el índice de Jaccard top-k y con la correlación de Spearman entre sus rankings.

3.6 Análisis estadístico

Como las métricas no siguen necesariamente una distribución normal y se observan sobre las mismas alertas, se emplean pruebas no paramétricas para muestras relacionadas (Demšar, 2006). Para $\Delta p@k$, $NC80$ y $CT3$ se aplica la prueba de Friedman:

$$\chi_F^2 = \frac{12n}{q(q+1)}\left[\sum_{j=1}^{q}\bar{R}_j^{\,2} - \frac{q(q+1)^2}{4}\right] \tag{7}$$

donde $q = 3$ es el número de condiciones y $\bar{R}_j$ el rango medio de la condición j; el tamaño del efecto es la W de Kendall, $W = \chi_F^2/[n(q-1)]$. Como la inversión de la decisión es dicotómica en cada alerta, se contrasta con la prueba Q de Cochran. Las comparaciones post hoc (C1 frente a C0, C2 frente a C0 y C1 frente a C2) usan la prueba de Wilcoxon de rangos con signo, o la de McNemar para $TID@k$, con la corrección de Holm: se ordenan los m valores p y se rechaza $H_{(i)}$ mientras $p_{(i)} \leq \alpha/(m-i+1)$ (Holm, 1979). El tamaño del efecto se reporta como $r = Z/\sqrt{n}$ (Fritz et al., 2012). H.E.2 se contrasta con la prueba de Wilcoxon de una muestra, unilateral, frente al valor de referencia 0,80. En todos los casos, $\alpha = 0{,}05$.

4. Desarrollo

Los modelos se organizan en tres niveles jerárquicos: (a) el detector, que asigna a cada alerta una familia de malware; (b) los explicadores, que asocian a cada alerta un vector de atribuciones; y (c) el evaluador, que mide la calidad de esas explicaciones.

4.1 Detector: Random Forest

Random Forest combina T árboles de decisión entrenados sobre muestras bootstrap, con un subconjunto aleatorio de características en cada división (Breiman, 2001). En la implementación de scikit-learn, la probabilidad de cada familia c es el promedio de las probabilidades estimadas por los árboles:

$$p(c \mid \mathbf{x}) = \frac{1}{T}\sum_{t=1}^{T} p_t\,(c \mid \mathbf{x}), \qquad \hat{c} = \operatorname*{argmax}_c\, p(c \mid \mathbf{x}) \tag{8}$$

Cada división minimiza la impureza de Gini $G(v) = 1 - \sum_c p_{v,c}^2$. Se usaron $T = 100$ árboles con profundidad máxima de 20, para acotar el costo de TreeSHAP sin sacrificar desempeño, y pesos de clase balanceados en cada muestra bootstrap. La condición de línea base (C0) es la importancia MDI, que acumula la reducción ponderada de impureza que produce cada característica j en los nodos v que la usan para dividir:

$$MDI_j = \frac{1}{T}\sum_{t=1}^{T} \sum_{v \in t:\, v(v)=j} \frac{N_v}{N}\, \Delta G(v) \tag{9}$$

donde N_v es el número de muestras que alcanzan el nodo y N el total. MDI es una medida global: asigna el mismo vector a todas las alertas y puede sesgarse hacia variables con muchos valores posibles (Strobl et al., 2007), lo que limita su utilidad para explicar casos individuales.

4.2 Explicador 1: SHAP (TreeSHAP)

SHAP explica cada predicción con el valor de Shapley de la teoría de juegos cooperativos (Shapley, 1953), que reparte entre las d características la diferencia entre la predicción y su valor esperado (Lundberg & Lee, 2017):

$$\phi_j(\mathbf{x}) = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\;(d - |S| - 1)!}{d!} [f_{\mathbf{x}}(S \cup \{j\}) - f_{\mathbf{x}}(S)] \tag{10}$$

donde F es el conjunto de características y $f_{\mathbf{x}}(S) = \mathbb{E}[f(\mathbf{X}) \mid \mathbf{X}_S = \mathbf{x}_S]$ es la predicción esperada cuando solo se conocen las características de S. La explicación resultante es un modelo aditivo que cumple la precisión local:

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^{d} \phi_j\,(\mathbf{x}), \qquad \phi_0 = \mathbb{E}[f(\mathbf{X})] \tag{11}$$

Junto con las propiedades de ausencia y consistencia, la precisión local hace de SHAP la única atribución aditiva con esas tres garantías (Lundberg & Lee, 2017). El cálculo directo de (10) crece exponencialmente con d; TreeSHAP lo resuelve de forma exacta para ensambles de árboles en tiempo $O(TLD^2)$, con L hojas por árbol y profundidad D (Lundberg et al., 2020). Se empleó TreeExplainer en la modalidad *tree_path_dependent*, que no requiere conjunto de referencia y es determinista, sobre la probabilidad de la familia predicha.

4.3 Explicador 2: LIME

LIME aproxima el modelo en la vecindad de la alerta con un modelo interpretable g, normalmente lineal (Ribeiro et al., 2016):

$$\xi(\mathbf{x}) = \operatorname*{argmin}_{g \in G}\; \mathcal{L}(f, g, \pi_{\mathbf{x}}) + \Omega(g) \tag{12}$$

$$\mathcal{L}(f, g, \pi_{\mathbf{x}}) = \sum_{\mathbf{z} \in Z} \pi_{\mathbf{x}}\,(\mathbf{z})[f(\mathbf{z}) - g(\mathbf{z}')]^2, \qquad \pi_{\mathbf{x}}(\mathbf{z}) = \exp\left(-\frac{D(\mathbf{x}, \mathbf{z})^2}{\sigma^2}\right) \tag{13}$$

donde Z contiene 5 000 muestras perturbadas alrededor de $\mathbf{x}$, $\mathbf{z}'$ es su representación interpretable (características discretizadas por cuartiles), $\pi_{\mathbf{x}}$ es un núcleo exponencial de proximidad con ancho $\sigma = 0{,}75\sqrt{d}$ (valor por defecto de LimeTabularExplainer) y $\Omega(g)$ penaliza la complejidad de g. Los coeficientes de g para la familia predicha se toman como

el vector $\boldsymbol{\phi}(\mathbf{x})$. Para medir la parsimonia sobre la atribución completa se recuperan los d coeficientes, aunque al analista solo se le presentan los k principales. Debido a su muestreo aleatorio, LIME puede producir explicaciones distintas para una misma alerta, lo que motiva la métrica EE.

4.4 Integración en el marco

El Algoritmo 1 resume la integración de los tres niveles. Para cada alerta, el marco devuelve un informe con la familia predicha y su probabilidad, las k características principales de cada explicador con su signo y su valor observado, y las métricas de calidad asociadas.

Algoritmo 1. Generación y evaluación de explicaciones por alerta

Entrada: detector f; alertas muestreadas $\{\mathbf{x}_i\}_{i=1}^{n}$; $k = 5$; $R = 10$

1. Calcular MDI (9) una sola vez y obtener T_k^{C0}.

2. Para cada alerta $\mathbf{x}_i$: (a) $\boldsymbol{\phi}^{C1} \leftarrow$ TreeSHAP$(f, \mathbf{x}_i)$ y registrar su tiempo; (b) para $r = 1, \ldots, R$: $\boldsymbol{\phi}^{C2,r} \leftarrow$ LIME$(f, \mathbf{x}_i, r)$ y registrar su tiempo; (c) para cada condición $c \in \{C0, C1, C2\}$, calcular $\Delta p@k$ y $TID@k$ (1–2), RP (4), $NC80$ y $CT3$ (5–6); (d) calcular EE (3) para C2 y la concordancia C1–C2.

3. Aplicar las pruebas de la Sección 3.6 y exportar el registro de observación estructurada.

Salida: registro por alerta y estadísticos de contraste.

5. Resultados

5.1 Desempeño del detector

El Random Forest se entrenó en 6,9 s y alcanzó una exactitud de 0,886, un F1 macro de 0,553 y una TFP macro de 0,162 (Tabla 3). El desempeño es alto para Mirai y bajo para las familias minoritarias: el 52,3 % de las ventanas de DarkNexus y el 19,2 % de las de Gafgyt se clasificaron como Mirai, lo que explica la TFP de 0,425 de esta familia (Figura 2). Las características con mayor importancia global (C0) fueron Rate (0,440), Telnet (0,148), Time_To_Live (0,050), AVG (0,037) y Tot size (0,029). En la muestra de 390 alertas, el detector acertó la familia en el 98,5 % de las de Mirai y en el 30,8 % de las de DarkNexus y de Gafgyt.

Tabla 3

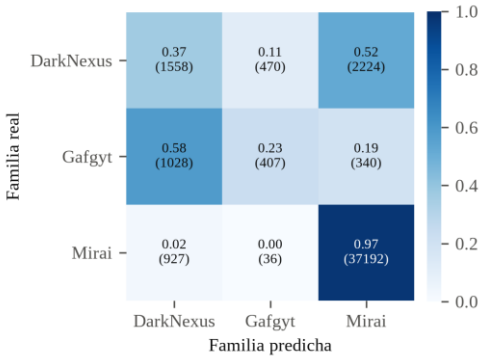
Desempeño del detector Random Forest en el conjunto de prueba

Familia	Precisión	Exhaustividad	F1	TFP	Ventanas
DarkNexus	0,443	0,366	0,401	0,049	4 252
Gafgyt	0,446	0,229	0,303	0,012	1 775
Mirai	0,936	0,975	0,955	0,425	38 155
Macro	0,608	0,523	0,553	0,162	44 182

Nota. Elaborado con src/03_entrenar_detector.py (results/tablas/desempeno_detector.csv).

Figura 2

Matriz de confusión normalizada por familia real



5.2 Calidad de las explicaciones

La Tabla 4 resume las métricas y la Figura 3 su distribución. En fidelidad, las tres condiciones fueron casi iguales: aunque Friedman detectó diferencias en $\Delta p@k$ ($p < 0{,}001$), el efecto fue muy pequeño ($W = 0{,}07$), ni TreeSHAP ($p_{Holm} = 1{,}00$) ni LIME ($p_{Holm} = 0{,}152$) superaron a C0, y la inversión no difirió ($Q = 3{,}5$; $p = 0{,}174$). En estabilidad, TreeSHAP fue determinista ($EE = 1$), pero su robustez quedó en 0,667, y LIME obtuvo $EE = 0{,}504$ y $RP = 0{,}429$; ninguna mediana alcanzó 0,80. En parsimonia, las diferencias fueron grandes ($W = 0{,}62$ y $0{,}52$): TreeSHAP concentró el 80 % de la atribución en 7 características, frente a 9 de C0 ($p_{Holm} < 0{,}001$; $r = -0{,}70$), mientras que LIME necesitó 12.

Tabla 4

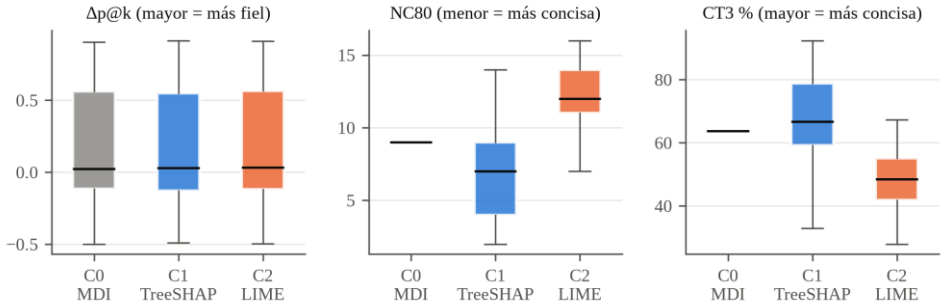
Métricas de interpretabilidad por condición y contrastes estadísticos (n = 390 alertas)

Métrica	C0	C1	C2	Prueba global	C1 vs C0	C2 vs C0
Δp@k	0,024	0,031	0,033	χ^2 = 58,4*; W = 0,07	r = −0,21	r = 0,08
TID@k (%)	41,0	41,8	41,3	Q = 3,5; p = 0,174	p = 0,75	p = 1,00
EE	1,000	1,000	0,504	—	—	p = 1,00
RP	1,000	0,667	0,429	—	p = 1,00	p = 1,00
NC80	9	7	12	χ^2 = 482,7*; W = 0,62	r = −0,70*	r = 0,86
CT3 (%)	63,8	66,7	48,5	χ^2 = 405,2*; W = 0,52	r = 0,26*	r = −0,87

Nota. Medianas, salvo TID@k. C0 = MDI; C1 = TreeSHAP; C2 = LIME. χ^2 de Friedman y W de Kendall; post hoc de Wilcoxon unilateral (r) o McNemar exacto (TID@k) con p ajustado por Holm; para EE y RP, Wilcoxon de una muestra frente a 0,80. C0 es global e invariante (EE = RP = 1). * $p_{Holm} < 0{,}001$. Elaborado con src/05_analisis_estadistico.py (results/tablas/contrastes.csv).

Figura 3

Distribución de la fidelidad y la parsimonia por condición



5.3 Costo y concordancia de los explicadores

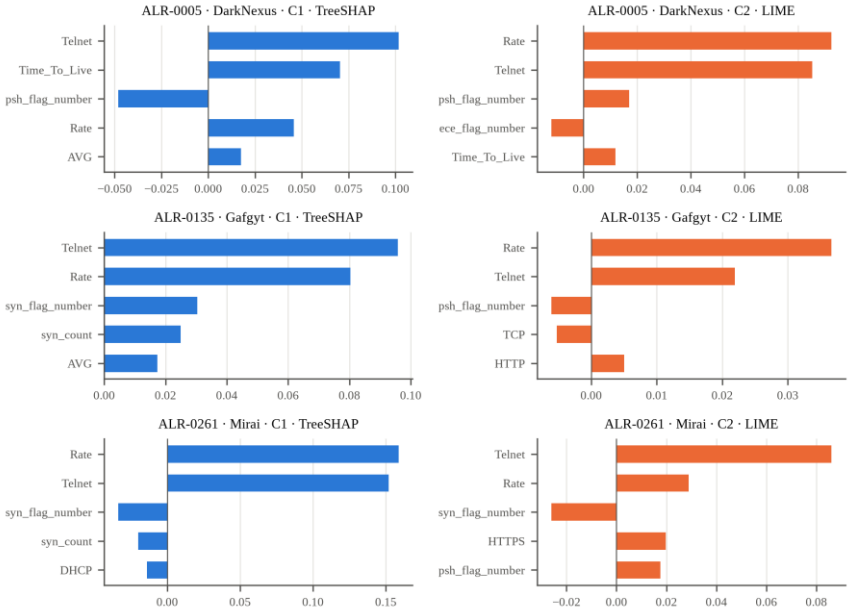
La mediana del tiempo por alerta fue de 244 ms para TreeSHAP y de 119 ms por ejecución para LIME, que con $R = 10$ repeticiones suma alrededor de 1,2 s por alerta. La memoria pico de LIME fue de 8,5 MB; la de TreeSHAP no pudo medirse con `tracemalloc`, porque su núcleo en C++ reserva memoria fuera del intérprete. La concordancia entre ambos métodos fue baja: la mediana del índice de Jaccard entre sus top-5 fue de 0,25 y la de la correlación de Spearman de 0,42. Aun así, ambos coincidieron en las dos características más frecuentes: Rate y Telnet aparecieron en el top-5 de 375 y 365 alertas con TreeSHAP y en las 390 con LIME.

5.4 Ejemplo de explicación para el analista

La Figura 4 muestra una alerta bien clasificada de cada familia. En las tres, ambos métodos señalan la tasa de paquetes (Rate) y el tráfico Telnet, coherentes con la propagación por fuerza bruta de credenciales en el puerto 23 típica de estas botnets (Antonakakis et al., 2017). TreeSHAP añade rasgos específicos de cada alerta, como Time_To_Live en DarkNexus o las banderas SYN en Gafgyt y Mirai, mientras que LIME distribuye el peso restante entre banderas y protocolos de baja atribución (HTTP, HTTPS, TCP), lo que ilustra su menor parsimonia.

Figura 4

Cinco características principales de una alerta por familia según TreeSHAP y LIME



6. Discusión

Los resultados confirman que ningún método es superior en todas las dimensiones (Arreche et al., 2024; Munilla & Khammas, 2026). TreeSHAP, exacto y determinista para árboles, fue el más conciso, lo que respalda su uso en detectores IoT basados en árboles (Le et al., 2022). La inestabilidad de LIME ($EE = 0{,}50$) se explica por su muestreo aleatorio de vecinos (Ribeiro et al., 2016), justifica medir la estabilidad con Jaccard (Hermosilla et al., 2025) y contrasta con la mayor estabilidad de ranking reportada por Munilla y Khammas (2026), quizá porque aquí 5 000 vecinos sobre 35 características discretizadas reordenan con facilidad los atributos de peso similar. Incluso TreeSHAP perdió parte de su top-5 ante perturbaciones de $0{,}01\sigma$ ($RP = 0{,}67$), porque los umbrales de los árboles generan cambios discretos en las contribuciones.

La falta de ventaja en fidelidad tiene dos explicaciones. Primero, el detector se apoya en pocas características dominantes (Rate y Telnet concentran el 59 % de la importancia global), de modo que el top-5 global ya contiene casi siempre lo que las explicaciones locales señalan. Segundo, la métrica se saturó: al neutralizar con la mediana de entrenamiento, dominada por Mirai, ninguna alerta predicha como Mirai cambió de familia en ninguna condición, mientras que en las predichas como DarkNexus o Gafgyt la decisión se invirtió en el 98–100 % de los casos con cualquiera de los tres métodos. Esta saturación es la causa de que TID@k sea casi idéntico (41 %) y muestra que la fidelidad por eliminación depende del valor de referencia elegido, un aspecto que los marcos de evaluación previos no suelen reportar (Arreche et al., 2024).

Para un SOC, conviene usar TreeSHAP como explicación por defecto en detectores de árboles (lista corta y reproducible en menos de 0,3 s), y LIME solo como segunda opinión agregada sobre varias ejecuciones. Limitaciones: solo usa la partición ARM y la modalidad de red; Gafgyt tiene cinco binarios, lo que reduce la generalización a esa familia; el detector es débil para las familias minoritarias, por lo que varias alertas explicadas son errores del modelo; y la memoria de TreeSHAP no se pudo medir. Como trabajo futuro se propone usar valores de referencia condicionados a la clase o distribuciones de fondo para la fidelidad, agregar varias ejecuciones de LIME, incorporar las cuatro arquitecturas y las modalidades SAR y STRACE, y evaluar las explicaciones con analistas de un SOC.

7. Conclusiones

El marco integró TreeSHAP y LIME a un detector Random Forest de malware IoT y evaluó 390 alertas del dataset internacional CIC-YNU-IoTMal 2026 frente a una línea base global. Respecto a O.E.1, no se acepta H.E.1: ninguno de los dos métodos mejoró la fidelidad frente a C0, en parte por la dominancia de dos características y la saturación de la métrica. Respecto a O.E.2, no se acepta H.E.2: TreeSHAP es reproducible entre ejecuciones, pero ninguna condición alcanzó una robustez de 0,80, y LIME fue inestable. Respecto a O.E.3, se acepta parcialmente H.E.3: TreeSHAP redujo de 9 a 7 las características necesarias para explicar el 80 % de la decisión, mientras que LIME la volvió más dispersa. En consecuencia, el marco mejora la interpretabilidad de las alertas solo por medio de TreeSHAP y en la dimensión de parsimonia. El aporte principal es un protocolo reproducible, con código y datos públicos, que muestra que la elección del explicador debe basarse en mediciones y que la explicación local no reemplaza automáticamente a la información global de la que ya dispone el analista.

Dataset y código fuente

- Dataset: CIC-YNU-IoTMal 2026 (Dadkhah et al., 2026). Kaggle: https://www.kaggle.com/datasets/raneemababneh/iot-malware-network-traffic-dataset-cic-ynu-2026. Página oficial: https://www.unb.ca/cic/datasets/ynu-iot-2026.html
- Código fuente, datos procesados, resultados y PDF de las fuentes: https://github.com/berrocalb/XAI_SOC_IoT

Referencias

Alahmadi, B. A., Axon, L., & Martinovic, I. (2022). 99% false positives: A qualitative study of SOC analysts' perspectives on security alarms. En *Proceedings of the 31st USENIX Security Symposium* (pp. 2783–2800). USENIX Association.

Alani, M. M., & Miri, A. (2022). Towards an explainable universal feature set for IoT intrusion detection. *Sensors, 22*(15), 5690. https://doi.org/10.3390/s22155690

Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., Durumeric, Z., Halderman, J. A., Invernizzi, L., Kallitsis, M., Kumar, D., Lever, C., Ma, Z., Mason, J., Menscher, D., Seaman, C., Sullivan, N., Thomas, K., & Zhou, Y. (2017). Understanding the Mirai botnet. En *Proceedings of the 26th USENIX Security Symposium* (pp. 1093–1110). USENIX Association.

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. En *Proceedings of the 31st USENIX Security Symposium* (pp. 3971–3988). USENIX Association.

Arreche, O., Guntur, T. R., Roberts, J. W., & Abdallah, M. (2024). E-XAI: Evaluating black-box explainable AI frameworks for network intrusion detection. *IEEE Access, 12*, 23954–23988. https://doi.org/10.1109/ACCESS.2024.3365140

Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R., & Herrera, F. (2020). Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion, 58*, 82–115. https://doi.org/10.1016/j.inffus.2019.12.012

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Dadkhah, S., Okey, O. D., Maret, S. A., Lo, Y.-W., Firouzi, A., Kuki, R., Sasaki, T., Yoshioka, K., Ban, T., Ozawa, S., & Ghorbani, A. A. (2026). CIC-YNU-IoTMal: A comprehensive multilayer dataset for static and dynamic analysis of IoT malware behavior. *Information Systems, 140*, 102722. https://doi.org/10.1016/j.is.2026.102722

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research, 7*, 1–30.

Fritz, C. O., Morris, P. E., & Richler, J. J. (2012). Effect size estimates: Current use, calculations, and interpretation. *Journal of Experimental Psychology: General, 141*(1), 2–18. https://doi.org/10.1037/a0024338

Hermosilla, P., Berríos, S., & Allende-Cid, H. (2025). Explainable AI for forensic analysis: A comparative study of SHAP and LIME in intrusion detection models. *Applied Sciences, 15*(13), 7329. https://doi.org/10.3390/app15137329

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics, 6*(2), 65–70.

IBM Security. (2023). *Global security operations center study results* [Informe elaborado por Morning Consult].

Le, T.-T.-H., Kim, H., Kang, H., & Kim, H. (2022). Classification and explanation for intrusion detection system based on ensemble trees and SHAP method. *Sensors, 22*(3), 1154. https://doi.org/10.3390/s22031154

Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence, 2*(1), 56–67. https://doi.org/10.1038/s42256-019-0138-9

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. En *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates.

Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence, 267*, 1–38. https://doi.org/10.1016/j.artint.2018.07.007

Munilla, J., & Khammas, R. M. (2026). Evaluation of explainable artificial intelligence in IoT intrusion detection systems under DeepFool adversarial conditions. *Sensors, 26*(10), 2924. https://doi.org/10.3390/s26102924

Neto, E. C. P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., & Ghorbani, A. A. (2023). CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors, 23*(13), 5941. https://doi.org/10.3390/s23135941

Pa, Y. M. P., Suzuki, S., Yoshioka, K., Matsumoto, T., Kasama, T., & Rossow, C. (2016). IoTPOT: A novel honeypot for revealing current IoT threats. *Journal of Information Processing, 24*(3), 522–533. https://doi.org/10.2197/ipsjjip.24.522

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1135–1144). ACM. https://doi.org/10.1145/2939672.2939778

Samek, W., Binder, A., Montavon, G., Lapuschkin, S., & Müller, K.-R. (2017). Evaluating the visualization of what a deep neural network has learned. *IEEE Transactions on Neural Networks and Learning Systems, 28*(11), 2660–2673. https://doi.org/10.1109/TNNLS.2016.2599820

Shapley, L. S. (1953). A value for n-person games. En H. W. Kuhn & A. W. Tucker (Eds.), *Contributions to the theory of games* (Vol. 2, pp. 307–317). Princeton University Press.

Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics, 8*, 25. https://doi.org/10.1186/1471-2105-8-25

Vectra AI. (2023). *2023 state of threat detection* [Informe de investigación]. https://content.vectra.ai/hubfs/downloadable-assets/V_2023-State-of-Threat-Detection.pdf